

Тема урока: числовые типы данных

Целочисленный тип данных

Целые числа в Python представлены типом данных `int` (сокращение `int` происходит от слова `integer`). Для определения целого числа типа `int` используется последовательность цифр от 0 до 9.

Явно указанное численное значение в коде программы называется **целочисленным литералом**. Когда Python встречает целочисленный литерал, он создает объект типа `int`, хранящий указанное значение.

```
n = 17 # целочисленный литерал
m = 7  # целочисленный литерал
```

Целочисленный тип данных `int` используют не только потому, что он встречается в реальном мире, но и потому, что он естественным образом возникает при создании большинства программ.

Преобразование строки в целое число

Для преобразования строки в целое число, мы используем команду `int()`:

```
num = int(input()) # преобразование считанной строки в целое число
```

Для преобразования строки в целое число не обязательно использовать команду `input()`.

Следующий код преобразует строку `12345` в целое число:

```
n = int('12345') # преобразование строки в целое число
```



Если строка не является числом, то при преобразовании возникнет ошибка.

Целочисленные операторы

Язык Python предоставляет четыре основных арифметических оператора для работы с целыми числами (`+`, `-`, `*`, `/`), а также три дополнительных (`%` для остатка, `//` для целочисленного деления и `**` для возведения в степень).

Следующая программа демонстрирует все целочисленные операторы:

```

a = 13
b = 7

total = a + b
diff = a - b
prod = a * b
div1 = a / b
div2 = a // b
mod = a % b
exp = a ** b

print(a, '+', b, '=', total)
print(a, '-', b, '=', diff)
print(a, '*', b, '=', prod)
print(a, '/', b, '=', div1)
print(a, '//', b, '=', div2)
print(a, '%', b, '=', mod)
print(a, '**', b, '=', exp)

```

В результате работы такой программы будет выведено:

```

13 + 7 = 20
13 - 7 = 6
13 * 7 = 91
13 / 7 = 1.8571428571428572
13 // 7 = 1
13 % 7 = 6
13 ** 7 = 62748517

```



При обычном делении (/) получается число не являющееся целым. Деление на ноль приводит к ошибке.

Длинная арифметика

Отличительной особенностью языка Python является неограниченность целочисленного типа данных. По факту, размер числа зависит только от наличия свободной памяти на компьютере. Это отличает Python от таких языков как C++, C, C#, Java где переменные целого типа данных имеют ограничение. Например, в языке C# диапазон целых чисел ограничен от -2^{63} до $2^{63} - 1$.

```

atom = 10 ** 80 # количество атомов во вселенной
print('Количество атомов =', atom)

```

Результатом выполнения программы будет число с 81 цифрой:

[illegible]

Символ разделитель

Для удобного чтения чисел можно использовать символ подчеркивания:

```
num1 = 25_000_000
num2 = 25000000
print(num1)
print(num2)
```

Результатом выполнения такого кода будет:

250000000
250000000

Числа с плавающей точкой

Наравне с целыми числами в Python есть возможность работы с дробными (вещественными) числами. Так, например, числа $\frac{2}{3}$, $\sqrt{2}$, π – являются вещественными и целого типа `int` недостаточно для их представления.

Дробные (вещественные) числа в информатике называют **числами с плавающей точкой**.

Для представления чисел с плавающей точкой в Python используется тип данных `float`.

```
e = 2.71828 # литерал с плавающей точкой
pi = 3.1415 # литерал с плавающей точкой
```

В Python, когда вы используете операцию деления `/`, результат всегда будет числом с плавающей точкой, даже если оба числа являются целыми. Вот пример:

```
print(4 / 2) # Результат: 2.0, а не 2
```



В отличие от математики, где разделителем является запятая, в информатике используется точка при записи чисел с плавающей точкой.

Преобразование строки к числу с плавающей точкой

Для преобразования строки к числу с плавающей точкой мы используем команду

`float()` :

```
num = float(input()) # преобразование считанной строки в число с
плавающей точкой
```

Для преобразования строки к числу с плавающей точкой необязательно использовать команду `input()` .

Следующий код преобразует строку `1.2345` к числу с плавающей точкой:

```
n = float('1.2345') # преобразование строки к числу с плавающей точкой
```



Если строка не является числом, то при преобразовании возникнет ошибка.

Арифметические операторы

Язык Python предоставляет четыре основных арифметических оператора для работы с числами с плавающей точкой (`+`, `-`, `*`, `/`) и один дополнительный (`**` – для возведения в степень).

Следующая программа демонстрирует арифметические операторы:

```
a = 13.5
b = 2.0

total = a + b
diff = a - b
prod = a * b
div = a / b
exp = a ** b

print(a, '+', b, '=', total)
print(a, '-', b, '=', diff)
print(a, '*', b, '=', prod)
print(a, '/', b, '=', div)
print(a, '**', b, '=', exp)
```

В результате работы такой программы будет выведено:

```
13.5 + 2.0 = 15.5
13.5 - 2.0 = 11.5
13.5 * 2.0 = 27.0
13.5 / 2.0 = 6.75
13.5 ** 2.0 = 182.25
```



Деление на ноль приводит к ошибке.

Преобразование между int и float

Неявное преобразование. Любое целое число (тип `int`) можно использовать там, где ожидается число с плавающей точкой (тип `float`), поскольку при необходимости Python автоматически преобразует целые числа в числа с плавающей точкой.

Явное преобразование. Число с плавающей точкой нельзя неявно преобразовать в целое число. Для такого преобразования необходимо использовать явное преобразование с помощью команды `int()`.

```
num1 = 17.89
num2 = -13.56
num3 = int(num1)
num4 = int(num2)
print(num3)
print(num4)
```

Результатом выполнения такого кода будет:

```
17
-13
```

Обратите внимание, что преобразование чисел с плавающей точкой в целое производится с округлением в сторону нуля, то есть `int(1.7) = 1`, `int(-1.7) = -1`.



Не путайте операцию преобразования и округления. Для округления чисел с плавающей точкой используются дополнительные команды. О них расскажем позже.

Встроенные функции

Python включает много заранее определенных функций. Программист не видит их реализацию, она скрыта. Достаточно знать, как эти функции называются и что они делают.

Мы уже сталкивались с встроенными функциями:

- `print()` – вывести на экран;
- `input()` – считать с клавиатуры;
- `int()` – преобразовать к целому числу;
- `float()` – преобразовать к числу с плавающей точкой.

Рассмотрим три новые встроенные функции, которые используются достаточно часто при работе с числами.

Функции `min()` и `max()`

Для определения соответственно минимального или максимального значения используются функции `min()` и `max()`. Аргументов у этих функций может быть любое количество, главное, чтобы они все поддерживали между собой операцию сравнения (например, `float` и `int` поддерживают сравнение, а `float` и `str` – нет).

Приведённый ниже код:

```
a = max(3, 8, -3, 12, 9)
b = min(3, 8, -3, 12, 9)
c = max(3.14, 2.17, 9.8)
print(a)
print(b)
print(c)
```

ВЫВОДИТ:

```
12
-3
9.8
```

Функция `abs()`

Модулем (абсолютной величиной) положительного числа называется само число, модулем отрицательного числа называется противоположное ему число, модуль нуля – нуль. Модуль числа a обозначается $|a|$, для него имеет место равенство:

$$|a| = \begin{cases} a, & \text{если } a > 0 \\ 0, & \text{если } a = 0 \\ -a, & \text{если } a < 0 \end{cases}$$

Для нахождения модуля (абсолютной величины) числа в Python используется функция `abs()`.

Приведённый ниже код:

```
print(abs(10))
print(abs(-7))
print(abs(0))
print(abs(-17.67))
```

ВЫВОДИТ:

```
10
7
0
17.67
```



Обратите внимание, все три функции `max()`, `min()`, `abs()` работают как с целыми числами, так и с числами с плавающей точкой.

Тема урока: строковый тип данных

Строковый тип данных

Строковый тип данных, как и числовой, очень часто используется в программировании. В Python строковый тип данных имеет название `str` (сокращение от string — строка, струна, ряд).

Для создания строковой переменной (литерала), мы должны заключить необходимый текст в кавычки. В Python можно использовать как одинарные кавычки, так и двойные:

```
s1 = 'Python rocks!'
s2 = "Python rocks!"
```

Напомним, что по умолчанию, команда `input()` считывает именно строку текста:

```
s = input() # переменная s имеет строковый тип str
```

Для задания пустой строки, мы используем две кавычки одинакового типа:

```
s1 = '' # пустая строка
s2 = ' ' # строка состоящая из одного символа пробела
```

Не стоит путать пустую строку и строку состоящую из одного символа пробела. Это абсолютно разные строки.

Длина строки

Длиной строки называется количество символов из которых она состоит. Чтобы посчитать длину строки используем встроенную функцию `len()` (от слова length — длина).

Следующий программный код:

```
s1 = 'abcdef'
length1 = len(s1)           # считаем длину строки из переменной s1
length2 = len('Python rocks!') # считаем длину строкового литерала
print(length1)
print(length2)
```

выведет:

```
6
13
```



При подсчете длины строки считаются все символы, включая пробелы.

Преобразование чисел в строку

Для преобразования строки к числу мы использовали функции `int()` и `float()`. Для обратного преобразования, то есть из числа в строку мы используем функцию `str()`:

Рассмотрим следующий программный код:

```
num1 = 1777    # целое число
num2 = 17.77   # число с плавающей точкой
s1 = str(num1) # преобразовали целое число в строку '1777'
s2 = str(num2) # преобразовали число с плавающей точкой в строку '17.77'
```



Иногда работать со строками намного проще, чем с числами. Даже если в условии задачи сказано, что дается число, нам ничто не мешает работать с ним как со строкой.

Конкатенация строк

Строки, как и числа, можно складывать. Операция сложения строк называется **конкатенацией** или **сцеплением**.

Рассмотрим следующий программный код:

```
s1 = 'ab' + 'bc'
s2 = 'bc' + 'ab'
s3 = s1 + s2 + '!!'
print(s1)
print(s2)
print(s3)
```

Результатом выполнения такого кода будет:

```
abbc
bcab
abbcbcab!!
```

Операция сложения строк в отличие от операции сложения чисел не является [коммутативной](#), то есть, от перестановки мест слагаемых-строк результат меняется!

С помощью конкатенации строк можно эмулировать вывод данных, который раньше мы делали используя необязательные параметры `sep` и `end`. Следующие две строки кода делают одно и то же:

```
print('a', 'b', 'c', sep='*', end='!')
print() # переход на новую строку
print('a' + '*' + 'b' + '*' + 'c' + '!')
```

Результатом выполнения такого кода будет:

```
a*b*c!
a*b*c!
```


Умножение строки на число

В Python также можно умножать строку на число. Такой оператор повторяет строку указанное количество раз.

Рассмотрим следующий программный код:

```
s = 'Hi' * 4  
print(s)
```

Результатом выполнения такого кода будет:

```
HiHiHiHi
```

Оператор умножения строки на число (repetition) очень удобен на практике. Например, мы хотим распечатать строку состоящую из 75 символов `-`. Мы можем это сделать с помощью кода:

```
print('-' * 75)
```

Результатом выполнения такого кода будет:

```
-----  
---
```



Строки можно умножать на число, но нельзя умножать на строку.

Примечания

Примечание 1. Тройные кавычки в Python используются для многострочного (multiline) текста. Например,

```
text = '''Python is an interpreted, high-level, general-purpose  
programming language.  
Created by Guido van Rossum and first released in 1991, Python design  
philosophy emphasizes code readability with its notable use of  
significant whitespace.'''
```

Примечание 2. На первый взгляд может показаться странным, что можно использовать как одинарные, так и двойные кавычки, однако такой подход позволяет очень легко добавлять в строку нужные кавычки:

```
s1 = 'Мы можем использовать в одинарных кавычках двойные кавычки "Война  
и мир"'  
s2 = "Мы можем использовать в двойных кавычках одиночные кавычки 'Война  
и мир'"  
print(s1)  
print(s2)
```

Результатом выполнения такого кода будет:

```
Мы можем использовать в одинарных кавычках двойные кавычки "Война и мир"  
Мы можем использовать в двойных кавычках одиночные кавычки 'Война и мир'
```

Оператор in

В Python есть специальный оператор `in`, который позволяет проверить, что одна строка находится внутри другой.

Приведённый ниже код:

```
s = 'https://pygen.ru/'
if 'a' in s:
    print('Введенная строка содержит символ a')
else:
    print('Введенная строка не содержит символ a')
```

проверяет, содержится ли в переменной `s` символ `'a'`, и выводит:

```
Введенная строка не содержит символ a
```

Использование вместе с логическими операторами

Мы можем использовать оператор `in` вместе с логическим оператором `not`, например:

```
s = input()
if '.' not in s:
    print('Введенная строка не содержит символа точки')
```

С помощью оператора `in` мы можем упростить следующий код, проверяющий, что значение переменной `s` равно одному из пяти символов `'a'`, `'e'`, `'i'`, `'o'`, `'u'`:

```
if s == 'a' or s == 'e' or s == 'i' or s == 'o' or s == 'u':
    print('YES')
```

до вида:

```
if len(s) == 1 and s in 'aeiou':
    print('YES')
```

С помощью оператора `in` мы можем проверять наличие сразу нескольких символов в строке.

Приведённый ниже код:

```
s = 'Sigma'
print('a' in s)
print('z' in s)
```

выводит:

```
True
False
```

Точное вхождение

Оператор `in` проверяет, содержится ли одна строка в другой строке как **точная последовательность символов**. В обеих строках символы должны находиться в том же порядке друг относительно друга и не должны быть разделены другими символами, чтобы выражение с оператором `in` вернуло значение `True`.

Приведённый ниже код:

```
print('ab' in 'abc')
print('ac' in 'abc')
```

выводит:

```
True
False
```

Во втором случае выводится `False`, потому что строка `'abc'` не содержит последовательности символов `'ac'` (символы `'a'` и `'c'` идут не подряд, они разделены символом `'b'`).

Чувствительность к регистру

Проверка с использованием оператора `in` чувствительна к регистру.

Приведённый ниже код:

```
s = 'Alpha'
print('p' in s)
print('P' in s)
```

выводит:

```
True
False
```

Тема урока: модуль math

Модуль math

Модуль `math` – один из наиважнейших в Python. Этот модуль предоставляет обширный функционал для проведения вычислений с действительными числами.

Для использования этих функций в начале программы необходимо подключить модуль, что делается командой `import` :

```
import math  
  
# программный код
```

После подключения модуля мы можем использовать его функции. Пусть мы хотим:

- вычислить $\sqrt{2}$ (корень квадратный из двух);
- округлить число 3.8 до ближайшего целого числа вверх и вниз

Приведённый ниже код:

```
import math  
  
num1 = math.sqrt(2)      # вычисление квадратного корня из двух  
num2 = math.ceil(3.8)    # округление числа вверх  
num3 = math.floor(3.8)   # округление числа вниз  
  
print(num1)  
print(num2)  
print(num3)
```

ВЫВОДИТ:

```
1.4142135623730951  
4  
3
```

Особенности подключения модулей

Как можно заметить из примера выше, для вызова функции мы вынуждены указывать название модуля и символ точки. С другой стороны, если функции используются достаточно часто, то такой вызов (постоянное указание названия модуля и символа точки) может усложнить программу и сделать её менее читабельной. Для того чтобы не указывать название модуля и символ точки при вызове функций, мы пишем следующий код:

```
from math import *

num1 = sqrt(2)      # вычисление корня квадратного из двух
num2 = ceil(3.8)    # округление числа вверх
num3 = floor(3.8)   # округление числа вниз

print(num1)
print(num2)
print(num3)
```

Таким образом, подключение модуля следующим образом:

```
from math import *
```

позволяет не писать название модуля и символ точки. При таком способе подключения, импортируются **абсолютно все** функции модуля `math`.

Если нужно использовать только некоторые функции модуля, то мы можем импортировать только их следующим образом:

```
from math import sqrt, ceil
```

Теперь мы можем вызывать функции `sqrt()` и `ceil()` без префикса `math.`, однако мы не можем вызвать функцию `floor()`, так как она не подключена:

```
from math import sqrt, ceil

print(sqrt(25))
print(ceil(34.7))

print(floor(12.8)) # приведет к ошибке, так как функция floor не
подключена
```

Список функций модуля math

Список наиболее часто используемых функций модуля `math` :

Функция	Описание
Округления	
<code>int()</code>	Округляет число в сторону нуля
<code>round(x)</code>	Округляет число <code>x</code> до ближайшего целого. Если дробная часть числа равна 0.5, то число округляется до ближайшего четного числа (банковское округление)
<code>round(x, n)</code>	Округляет число <code>x</code> до <code>n</code> знаков после точки
<code>floor(x)</code>	Округляет число <code>x</code> вниз («пол»)
<code>ceil(x)</code>	Округляет число <code>x</code> вверх («потолок»)
<code>abs(x)</code>	Модуль числа <code>x</code> (абсолютная величина)
Корни, логарифмы, степени и факториал	
<code>sqrt(x)</code>	Квадратный корень числа <code>x</code>
<code>pow(x, n)</code>	Возведение числа <code>x</code> в степень <code>n</code>
<code>log(x)</code>	Натуральный логарифм числа <code>x</code> . Основание натурального логарифма равно числу <code>e</code>
<code>log10(x)</code>	Десятичный логарифм числа <code>x</code> . Основание десятичного логарифма равно числу 10

<code>log(x, b)</code>	Логарифм числа <code>x</code> по основанию <code>b</code>
<code>factorial(n)</code>	Факториал натурального числа <code>n</code>
Тригонометрия	
<code>degrees(x)</code>	Преобразует угол <code>x</code> , заданный в радианах, в градусы
<code>radians(x)</code>	Преобразует угол <code>x</code> , заданный в градусах, в радианы
<code>cos(x)</code>	Косинус угла <code>x</code> , задаваемого в радианах
<code>sin(x)</code>	Синус угла <code>x</code> , задаваемого в радианах
<code>tan(x)</code>	Тангенс угла <code>x</code> , задаваемого в радианах
<code>acos(x)</code>	Возвращает угол в радианах от 0 до π , <code>cos</code> которого равен <code>x</code>
<code>asin(x)</code>	Возвращает угол в радианах от $-\frac{\pi}{2}$ до $\frac{\pi}{2}$, <code>sin</code> которого равен <code>x</code>
<code>atan(x)</code>	Возвращает угол в радианах от $-\frac{\pi}{2}$ до $\frac{\pi}{2}$, <code>tan</code> которого равен <code>x</code>
<code>atan2(y, x)</code>	Полярный угол (в радианах) точки с координатами <code>(x, y)</code>



Для извлечения квадратного корня можно воспользоваться кодом `n ** 0.5`, вместо `math.sqrt(n)`.



Для извлечения квадратного корня можно воспользоваться кодом `n ** 0.5`, вместо `math.sqrt(n)`.

Список констант модуля math

Модуль `math` предоставляет ряд встроенных математических констант:

Константа	Описание
<code>pi</code>	Число $\pi = 3.141592653589793$
<code>e</code>	Число $e = 2.718281828459045$ (константа Эйлера)

Примечания

Примечание 1. Все функции модуля `math` возвращают значение, которое можно вывести на экран, присвоить другой переменной или использовать в математическом выражении.

Примечание 2. Для использования функций `int()`, `float()`, `abs()`, `min()`, `max()`, `round()` подключать модуль `math` нет необходимости. Это так называемые встроенные функции.

Примечание 3. Вызов функций `pow(x, n)` можно заменить использованием оператора возведения в степень: `x**n`.

Примечание 4. Импортировать функции (или любые другие объекты) из модуля можно несколькими способами:

```
from math import *  
  
num = sqrt(10)
```

```
from math import sqrt  
  
num = sqrt(10)
```

```
import math  
  
num = math.sqrt(10)
```

Однако рекомендуется избегать импортирования через `from math import *`, так как нет ясности, какие функции были импортированы и это загрязняет пространство имён лишними, неиспользуемыми функциями. Вместо этого рекомендуется использовать `from math import sqrt` или `import math`, чтобы явно указать, что именно вы импортируете. Это делает ваш код более читаемым и управляемым.